

Desenvolvimento Orientado a Testes (TDD) com Node.js



Conteudista: Prof. Esp. Jônatas Carvalho

Revisão Textual: Esp. Anna Carolina Guimarães

Objetivo da Unidade:

- Entendimento dos princípios do TDD e a aplicação de seu ciclo em projetos de desenvolvimento de software.

 [Material Teórico](#)

 [Referências](#)



Material Teórico

O Que é TDD e Pirâmide de Testes?

O *Test Driven Development* ou Desenvolvimento Orientado a Testes (TDD) é uma metodologia ágil que realiza a integração do desenvolvimento de software e a criação de testes automatizados de forma simultânea, dando prioridade na qualidade e na manutenção do código.

O TDD foi criado por Kent Beck em 2002 e descrito em seu livro *Test-Driven Development: By Example*. O surgimento dessa metodologia nasceu com a finalidade de enfrentar desafios como *bugs* que eram difíceis de detectar, sistemas de grande complexidade e necessidade de entendimento mais claro de requisitos de negócio.

No TDD, ao invés de primeiro iniciarmos o desenvolvimento do software e só, então, depois criar os testes, a metodologia propõe que primeiro os testes sejam elaborados antes de um código funcional, isso é interessante, pois garante que cada

funcionalidade que irá ser implementada tem base em uma necessidade explícita do sistema.

Os testes automatizados são fundamentais nesse processo, uma vez que possibilitam que seja verificado se o comportamento da aplicação corresponde ao esperado. Os testes automatizados, além de proporcionar eficiência e economia de tempo ao diminuir a necessidade de testes manuais, que são longos e suscetíveis a falhas humanas, asseguram consistência e fácil reprodução. Isso implica na observância estrita dos mesmos passos em cada execução, eliminando variações e garantindo maior confiabilidade. Além disso, os testes auxiliam na identificação antecipada de falhas, diminuindo o custo e o esforço necessários para sua correção, além de auxiliarem na manutenção do código, garantindo que mudanças não prejudiquem a funcionalidade já existentes.

Utilizar o TDD como metodologia no desenvolvimento de um software ajuda na resolução de problemas recorrentes de desenvolvimento, como regressões, que são falhas que podem ser ocasionadas por alterações no código fazendo com que a cobertura de código abaixo deixando essa inadequada e, assim, deixando partes do software sem cobertura e sem testes. Podemos citar outro benefício que é a redução da dependência do conhecimento humano, pois os testes automatizados são como uma documentação viva, que basicamente demonstra como cada unidade de código deve se comportar, ou seja, com a devida

entrada de dados deve ser produzida tal saída de dados, permitindo, assim, que as equipes possam passar por cima sem dificuldades a substituições de membros da equipe ou até mesmo a falta de uma documentação formal.

Existem diferentes níveis de testes em TDD:

- **Testes unitários:** os testes unitários ou testes de unidade (essa nomenclatura muda em diferentes fontes) são testes que realizam a verificação de pequenas partes ou unidades de nosso código, como funções, métodos ou condicionais, tudo isso realizando uma análise isolada desses trechos de código. Esses testes facilitam a busca de erros específicos, promovendo confiança na refatoração e estabelecendo contratos de interface consistentes;
- **Testes de integração:** estes avaliam como módulos ou componentes de um software realizam interação entre si, realizando a verificação no fluxo de dados, compatibilidade e dependências externas;
- **Testes *end to end*:** testes *end to end* ou e2e fazem uma simulação real de como o sistema deve se comportar em um cenário real sendo utilizado por um usuário, por isso possuem esse nome. Se fôssemos abrigar o nome desses testes, ficaria algo próximo a “testes de ponta a ponta”. Esses testes têm a finalidade de verificar o funcionamento completo do sistema.

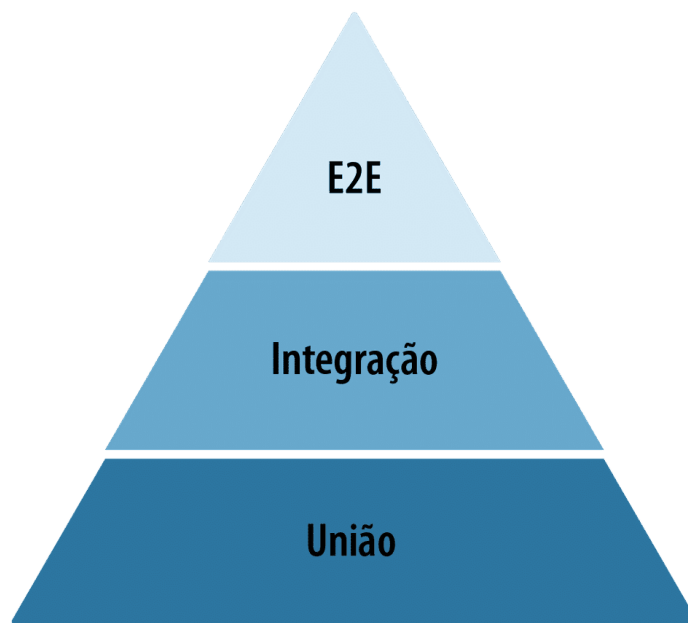


Figura 1 – Pirâmide de testes

#ParaTodosVerem: a imagem ilustra a pirâmide de testes, na qual sua base é composta de testes unitários, no meio ficam os testes de integração ou testes integrados e no topo da pirâmide estão os testes e2e ou testes *end to end*. Fim da descrição.

Segundo Kent Beck (2002), esses testes possuem uma hierarquia que é representada pela pirâmide de testes, no qual os testes unitários ficam na base da pirâmide, pois são eles que vão ser em maior quantidade, porém em contrapartida são os testes que possuem o menor custo. Os testes de integração ficam no meio dessa pirâmide, pois têm uma quantidade moderada e, também, um custo moderado. No topo dessa pirâmide, ficam os testes e2e que são em menor quantidade por sua complexidade de construção e são os de maior custo, pois já que vamos de fato reproduzir o comportamento de um usuário, diversos componentes, como instâncias da aplicação, banco de dados etc., devem estar funcionais para a realização desses testes.

Dificuldades, Benefícios, Onde Usar e Como Iniciar o Uso de TDD

O uso e a aplicação do TDD, apesar de ser altamente benéfico, está muito longe de ser simples em sua aplicação. No início de sua implantação, é muito comum enfrentar diversas dúvidas sobre por onde começar e quais tipos de testes devem ser priorizados na escrita. Há, também, uma impressão de que o TDD torna o processo de desenvolvimento mais lento, principalmente quando comparamos com métodos tradicionais, e de fato no começo se torna sim mais lento, mas o TDD traz um enorme amadurecimento que com o tempo pode se tornar tão eficaz que se torna sem dúvidas mais rápido. Além da velocidade, as entregas possuem uma qualidade superior por estarem pautadas em cima dos testes desenvolvidos previamente.

O TDD é guiado por algumas leis e padrões que irão estruturar o processo, por exemplo, um desenvolvedor só pode escrever código que irá atuar em um ambiente produtivo após a criação de um teste que falhe, garantindo que o que for desenvolvido faça esse teste parar de dar falha, ou seja, garante que o código irá atender uma necessidade do sistema. Outra prática muito utilizada está focada no desenvolvimento de código necessário apenas para fazer o teste atual passar, isso evita antecipações desnecessárias e mantém, assim, o foco em funcionalidades claras.

Como para tudo na vida, o TDD é uma ótima metodologia, mas não é uma bala de prata que pode resolver tudo, ou seja, ele não é adequado para todas as situações e/ou casos. Seu uso é mais eficaz em projetos que exigem uma refatoração contínua e um desenvolvimento com feedback rápido. E cenários que não devemos utilizar o TDD são: desenvolvimento exploratório (quando temos a necessidade de verificar se algo é possível de se fazer), protótipos (quando a empresa precisa mostrar algo para o cliente, mas não é totalmente funcional e tem um prazo curto para ser entregue) e quando a aplicação possui requisitos altamente voláteis (quando o cliente não sabe muito bem o que quer e muda algumas *features* no projeto).

Em resumo, o TDD é uma metodologia poderosa que proporciona às equipes que o utilizam uma melhoria na comunicação, na confiança e um desenvolvimento de qualidade. Mesmo com toda a adaptação inicial, seu uso transforma desenvolvedores em profissionais mais eficientes e preparados para enfrentar os problemas complexos que hoje a computação tenta resolver.

Princípios Básicos de TDD: *Red, Green e Refactor*

O TDD possui em seu ciclo três etapas que são fundamentais. Essas etapas, que são *Red, Green* e *Refactor*, podem ser iterativas e se acumulam. Não é obrigatória a implementação das 3 etapas, pois existem contextos específicos, como a prototipação e a experimentação em que não vamos encontrar a necessidade de

implementação de todas as etapas, porém Beck (2002) destaca que é fundamental a implementação de todo o ciclo, pois esse processo traz qualidade e uma diminuição de *bugs* garantindo ainda mais que o código esteja sempre alinhado aos requisitos que o sistema necessita.

Como já mencionado, o ciclo possui 3 etapas, que são:

- **Red:** essa etapa possui esse nome, pois vamos iniciar os códigos com um teste que falha, e o teste falha não porque contém algo errado, mas sim porque não possui código para ser testado, ou seja, esse teste vai ser escrito para a validação de uma funcionalidade desejada, que no caso ainda será desenvolvida futuramente. O comportamento esperado pelo sistema deve ser bem definido para que o teste reflita esses requisitos;
- **Green:** como o nome da etapa anterior remete a um teste que falha, a mesma coisa acontece nesta etapa, pois possui esse nome porque remete ao fato de escrever um código para que o teste desenvolvido anteriormente passe. É bom enfatizar que o código escrito deve estar focado em resolver o problema da etapa *Red*, devendo ser o mínimo quanto possível. A ideia aqui é o mínimo esforço, não focamos na solução perfeita, pois o objetivo do código é fazer com que o teste que falhava agora mude seu *status* para sucesso. Focando apenas na mudança do *status* do teste, evitamos distrações e garantimos a evolução de cada funcionalidade de forma incremental;

- **Refactor:** nesta última etapa do ciclo, vamos focar na melhoria do código que foi construído anteriormente. Aqui devemos realizar alterações para que nosso código fique mais genérico através de abstrações podendo deixá-lo reutilizável. Além disso, removeremos duplicações e código desnecessário, assim teremos um código com mais qualidade, mais legível e muito mais eficiente.

Em resumo o ciclo do TDD é bem simples, criar um teste que falha, escrever código para que o teste passe, refatorar para que o código fique mais simples e eficiente.

Frameworks de Teste

No mercado, temos 3 principais *frameworks* para testes em Node.js, são eles: *Mocha*, *Jasmine* e *Jest*.

Glossário

Frameworks: são um conjunto de ferramentas e bibliotecas que servem para facilitar o desenvolvimento de sistemas. São muito utilizados e praticamente todas as linguagens utilizadas no mercado de trabalho possuem *frameworks* que facilitam muito o trabalho.

Vídeo

Frameworks – Dicionário do Programador

Assista a seguir o trailer do filme indicado.



Em 2011, foi criado o *framework* Mocha que desde então sempre se manteve entre os mais populares da comunidade Javascript que é bem grande. Este *framework* possui uma grande quantidade de funcionalidades e é bem flexível em seu uso e, por ser um projeto já considerado antigo, possui muito conteúdo pela internet e um ótimo suporte disponível, um ponto negativo é que o Mocha não vem com uma biblioteca de asserção por padrão, devemos fazer sua instalação.

```

var assert = require('assert');
describe('Array', function () {
  describe('#indexOf()', function () {
    it('should return -1 when the value is not present', function () {
      assert.equal([1, 2, 3].indexOf(4), -1);
    });
  });
});

```

Criado em 2009 e publicado em 2010 pela *Pivotal Labs*, o Jasmine é um *framework* de testes muito utilizado em BDD – *Behavior-Driven Development* ou em português Desenvolvimento Guiado por Comportamento, que é uma metodologia ágil de desenvolvimento de *software* que trabalha em cima da colaboração constante entre desenvolvedores, *testers* e *stakeholders*. A comunidade do Jasmine dentre os três abordados em nosso material é a menor de todas, porém – por ser já bem antiga e madura – possui muito suporte e materiais para consulta. A execução de testes no Jasmine possui uma velocidade razoável e *mocks*, *spies* e *asserts* já vêm juntos com o *framework*.

```

describe("A suite is just a function", function() {
  let a;
  it("and so is a spec", function ( ) {
    a = true;
    expect(a) . toBe(true);
  });
});

```

Desenvolvido pelo Facebook, o *framework Jest* é bem completo para testes em aplicações *front-end* e *back-end*. Criado em 2014 como um projeto *open source*, é um *framework* focado na facilidade de uso e em sua simplicidade. O *Jest* já vem com a biblioteca de asserções em seu pacote principal. Por ser um projeto mais recente,

possui um número menor de ferramentas disponíveis, porém por ser muito utilizado em *React* com o passar do tempo isso não será um problema.

Refleta

Sugiro que faça uma pesquisa mais rebuscada sobre *Mocha*, *Jasmine* e *Jest* e, depois, faça um exercício: crie uma aplicação e responda: qual dos 3 *frameworks* você utilizaria e por quê?

Os três *frameworks* são ótimos e podem ser utilizados sem medo de falta de suporte, atualizações ou algo do tipo. Por ser usado muito em *frameworks* como *Node.js*, *React*, *Angular* e *Vue*, decidi utilizar o *Jest* para realizar um teste mais apropriado e fornecer assim uma visão mais aprofundada e densa de como funcionam os testes em uma aplicação real.

Utilizando o TDD em um Projeto Utilizando Node.js e Jest

Lembram da aplicação Conversor de Números Decimais para Números Binários que construímos em nossa primeira aula? Bom, que tal dessa vez criarmos a aplicação utilizando a metodologia TDD?

Seguiremos três passos principais do ciclo TDD: *Red*, *Green* e *Refactor*, ou seja, vamos escrever um teste que falha, implementar o código para que o teste passe e refatorar o código para que melhore sua qualidade.

Vamos começar escrevendo o teste *Red*. Para isso, basta criar um arquivo chamado `app.test.js` e inserir o seguinte código:

```
const request = require("supertest");
const express = require("express");
const app = require("./app");

describe("Decimal to Binary API", () => {
  it("deve retornar um número binário se o número informado for um decimal válido", () => {
    const response = await request(app).get("/to-binary/10");
    expect(response.status).toBe(200);
    expect(response.body).toEqual({ decimal: 10, binary: "1010" });
  });

  it("deve retornar erro 400 caso for informado um número decimal inválido", () => {
    const response = await request(app).get("/to-binary/invalid");
    expect(response.status).toBe(400);
    expect(response.body).toEqual({ error: "Número decimal inválido" });
  });
});
```

O teste acima faz duas verificações e possui algumas etapas, a primeira é fazer uma chamada para um *endpoint* ("`/to-binary/10`") passando o valor 10 como parâmetro. Após essa etapa, ele espera receber um status http 200 e um *json* com o resultado da conversão correta, já que 10 é um número decimal válido. Em sequência, ele faz uma nova chamada para o mesmo *endpoint* só que agora passando a palavra *invalid* e isso faz com que a função dê erro, pois só converte para binário os números decimais válidos; com isso, o teste espera receber um status http de erro 400 e um *json* com a frase "Número decimal inválido".

Após escrever o código acima, vamos instalar o *express* e, então, rodar o *Jest* para executar o teste. Para isso, basta digitar o seguinte código em seu terminal (uma linha por vez):

- `npm install express;`
- `npx jest.`

Após a execução desses comandos, você receberá uma saída semelhante a esta em seu terminal:

```
+ npx jest
FAIL ./app.test.js
  Decimal to Binary API
    ✕ deve retornar um número binário se o número informado for um decimal válido (3 ms)
    ✕ deve retornar erro 400 caso for informado um número decimal válido (1 ms)

● Decimal to Binary API > deve retornar um número binário se o número informado for um decimal válido

TypeError: app.address is not a function

   6 | describe("Decimal to Binary API", () => {
   7 |   it("deve retornar um número binário se o número informado for um decimal válido", async () => {
>  8 |     const response = await request(app).get("/to-binary/10");
      |                                     ^
   9 |     expect(response.status).toBe(200);
  10 |     expect(response.body).toEqual({ decimal: 10, binary: "1010" });
  11 |   });

at Test.serverAddress (node_modules/supertest/lib/test.js:46:22)
at new Test (node_modules/supertest/lib/test.js:34:14)
at Object.obj.<computed> [as get] (node_modules/supertest/index.js:43:18)
at Object.get (app.test.js:8:41)

● Decimal to Binary API > deve retornar erro 400 caso for informado um número decimal válido

TypeError: app.address is not a function

  12 |
  13 |   it("deve retornar erro 400 caso for informado um número decimal válido", async () => {
> 14 |     const response = await request(app).get("/to-binary/invalid");
      |                                     ^
  15 |     expect(response.status).toBe(400);
  16 |     expect(response.body).toEqual({ error: "Invalid decimal number" });
  17 |   });

at Test.serverAddress (node_modules/supertest/lib/test.js:46:22)
at new Test (node_modules/supertest/lib/test.js:34:14)
at Object.obj.<computed> [as get] (node_modules/supertest/index.js:43:18)
at Object.get (app.test.js:14:41)

Test Suites: 1 failed, 1 total
Tests:       2 failed, 2 total
Snapshots:   0 total
Time:        1.634 s, estimated 2 s
Ran all test suites.
```

Figura 2 – Saída de erro no terminal após a execução do teste criado

Fonte: Reprodução

#ParaTodosVerem: a imagem ilustra o teste que acabamos de criar falhando. Isso acontece, pois ele está testando um código que ainda não existe e será criado na próxima etapa do TDD, ele informa que os dois testes falharam e o tempo de execução dos testes. Fim da descrição.

Isso indica que o teste falhou.

Agora, vamos para a etapa *Green*, na qual vamos implementar o código. Basicamente vou utilizar o mesmo código que usamos anteriormente, porém vamos fazer algumas pequenas modificações. Vamos criar um arquivo chamado `app.js`, dentro desse arquivo vamos inserir o seguinte código:

```
const express = require("express");

const app = express();

// Endpoint para converter decimal para binário
app.get("/to-binary/:decimal", (req, res) => {
  const decimal = parseInt(req.params.decimal, 10);

  if (isNaN(decimal)) {
    return res.status(400).json({ error: "Número decimal inválido" });
  }

  const binary = decimal.toString(2);
  res.json({ decimal, binary });
});

module.exports = app;
```

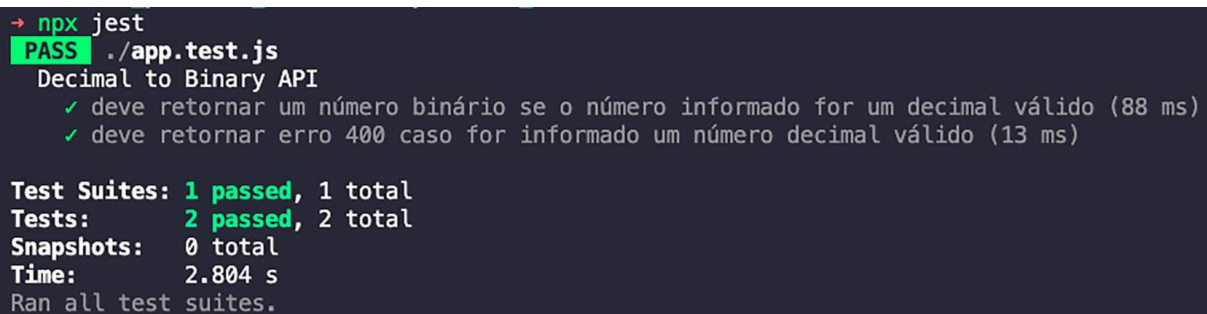
Agora, crie o arquivo `index.js` e insira o seguinte código:

```
const app = require("../app");
const port = 3000;

app.listen(port, () => {
  console.log(`App running on http://localhost:${port}`);
});
```

Fizemos essa separação para que no arquivo `index.js` fiquem apenas as configurações da requisição e, no arquivo `app.js`, fique somente a lógica que faz a conversão do número decimal para número binário.

Como concluímos a segunda parte, vamos novamente executar o teste com o comando que foi passado anteriormente, mas agora vamos receber a seguinte saída:



```
→ npx jest
PASS ./app.test.js
  Decimal to Binary API
    ✓ deve retornar um número binário se o número informado for um decimal válido (88 ms)
    ✓ deve retornar erro 400 caso for informado um número decimal válido (13 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        2.804 s
Ran all test suites.
```

Figura 3 – Saída exibindo que o teste passou

Fonte: Reprodução

#ParaTodosVerem: a imagem ilustra o teste que antes criamos passando com sucesso. Isso acontece, pois já realizamos a implementação do código que ele

testa. Ele mostra que a bateria de testes passou, dois testes foram executados e demorou 2.804 segundos para a execução desses testes. Fim da descrição.

Agora, vamos implementar a última fase que é a *Refactor*. Nosso código por ser um código pequeno e simples não há muito no que podemos mexer de forma que fique mais eficiente, mas podemos utilizar um *middleware* que é uma boa prática.

Glossário

Middleware: é uma camada do software que faz a intermediação entre a requisição realizada pelo usuário/cliente e a resposta da aplicação. É muito utilizado em *frameworks web* como o Express.js, porém esse conceito não fica apenas na camada web, mas sim em outros contextos.

Nosso *middleware* vai entrar em nosso código para validar os números de uma forma mais genérica, pois poderíamos ter adição de novos *endpoints* com novas conversões, por exemplo. Em nossa refatoração, vamos criar o arquivo `validateDecimal.js`

no mesmo diretório em que está o arquivo app.js e, dentro dele, vamos inserir o seguinte código:

```
const validateDecimalParam = (req, res, next) => {
  const decimal = parseInt(req.params.decimal, 10);
  if (isNaN(decimal)) {
    return res.status(400).json({ error: "Número decimal inválido" });
  }
  req.decimal = decimal;
  next();
};
module.exports = validateDecimalParam;
```

E o nosso arquivo app.js deve ficar da seguinte maneira:

```
const express = require("express");
const validateDecimalParam = require("./validateDecimal");

const app = express();

app.get("/to-binary/:decimal", validateDecimalParam, (req, res) => {
  const binary = req.decimal.toString(2);
  res.json({ decimal: req.decimal, binary });
});

module.exports = app;
```

Após isso podemos novamente rodar o comando **`npm jest`** e, então, você verá que, mesmo com essa refatoração e a adição de mais um arquivo, os testes não falham. Perceba que de fato seguir essas três etapas usando o TDD como metodologia de desenvolvimento pode parecer um trabalho mais extenso e demorado. De fato, essa é a primeira impressão que temos, mas perceba, também, que todo o código está coberto pelos testes criados e, além disso, nosso código é de extrema qualidade, fato que é muito importante. Com isso, como já citado, mesmo parecendo ser mais lento, após adquirir experiência entenderemos o quão eficiente é utilizar o TDD e, também, entenderemos a qualidade que ele traz para nosso código final.



Referências

BECK, K. *Test-driven development: by example*. Boston: Addison-Wesley, 2002.

JASMINE. *Jasmine Documentation*. Disponível em: <<https://jasmine.github.io/>>. Acesso em: 28/12/2024.

JEST. *Jest Documentation*. Disponível em: <<https://jestjs.io/docs>>. Acesso em: 28/12/2024.

MOCHA. *Mocha Documentation*. Disponível em: <<https://mochajs.org/>>. Acesso em: 28/12/2024.

ROLT, M. O que é a pirâmide de testes? *One Day Testing blog*. [S.d.]. Disponível em: <<https://blog.onedaytesting.com.br/piramide-de-teses/>>. Acesso em: 28/12/2024.